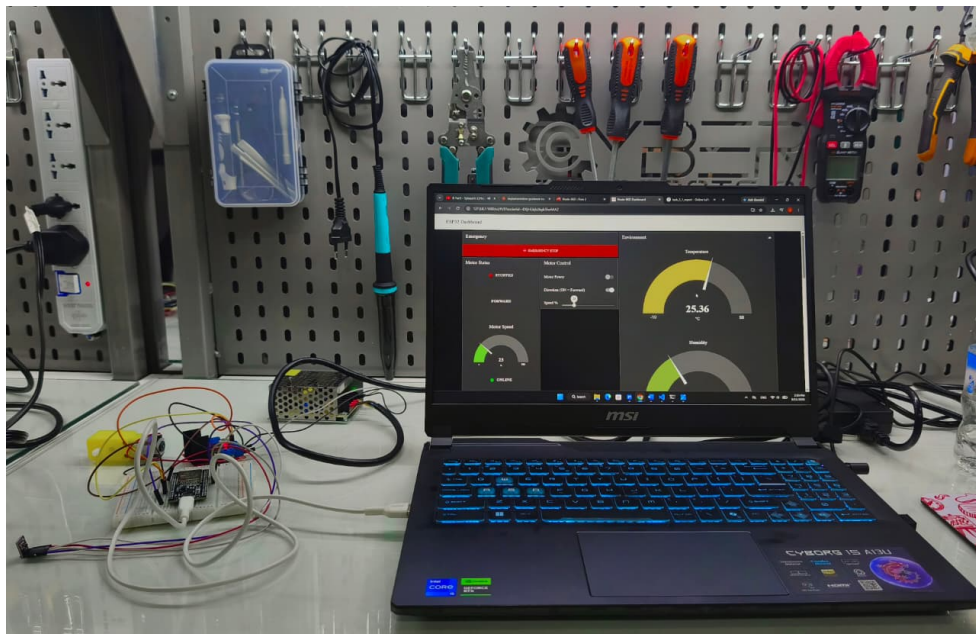


Firestore Realtime Database Cloud Logging for a Local ESP32 MQTT System

Extending the Task 3.1 Node-RED Dashboard with Live State Mirroring, Timestamped History, and Reused Availability Detection



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents Task 3.3: extending the Task 3.1 Node-RED dashboard so that live sensor readings and motor state also land in a Firebase Realtime Database, giving the system a cloud record on top of the local dashboard. No new hardware, wiring, or MQTT subscription was introduced – the entire Firebase branch is additive, tapping off nodes that already exist in the Task 3.1 flow.

Two Firebase writers were built. A **current-state** writer overwrites a single `/current` node in the database every time any tracked value changes, so it always reflects “right now.” A **history** writer snapshots that same state once every 15 seconds and pushes it as a new, uniquely-keyed record under `/history`, building an append-only log over time. Both writers deliberately reuse the same validated sensor values already driving the dashboard gauges, and the same dual-signal (LWT + watchdog) **Connection monitor** output already built in Task 3.1 – rather than re-deriving either from scratch.

The most instructive part of this task was a data-sharing bug that produced no error at all: two separate **function** nodes need to agree on where cached state lives, and Node-RED’s default **context** scope is private to each node. The history branch silently wrote nothing for a period of testing, with no red error triangle anywhere to flag it – exactly the kind of “looks fine, does nothing” failure mode Task 3.1’s own debugging journal had already warned about. Section walks through how it was found and fixed.

2. System Overview & Goal

This task adds a cloud-logging branch on top of the existing Task 3.1 Node-RED flow and its underlying Task 2.7 hardware – nothing about the ESP32 firmware, wiring, or MQTT broker changed:

```
(BME280, I2C) --> [ESP32] --publish--> [Mosquitto 192.168.1.33:1883] --subscribe--> [Node-RED :1880/ui]
                                     Node-RED
                                     --Set /current, Push /history--> [Firebase Realtime Database]
                                     ESP32
--LWT (retained, on disconnect)--> [Mosquitto] --> device/esp32/status --> [Connection monitor] -->
                                     [Firebase /current/status]
```

- Firebase writes are a pure branch off nodes that already exist in the Task 3.1 flow – no second MQTT subscription, no duplicated validation logic, no duplicated connection-monitoring logic.
- The Node-RED dashboard built in Task 3.1 is completely unmodified: every existing wire, gauge, chart, and control still behaves exactly as before. Firebase nodes only ever *receive* messages branched off existing outputs; they never sit in series on the dashboard’s own data path.
- Two independent Firebase Realtime Database write patterns are used: **Set** (full overwrite) for live state, and **Push** (auto-keyed append) for history.

3. Hardware Architecture (Unchanged)

Task 3.3 adds no new wiring, sensors, or actuators – the ESP32, BME280, and L298N motor driver are exactly as wired and pinned in Task 2.7/3.1. The schematic and assembled circuit are included here for completeness, since the cloud-logging branch this report documents writes values produced by this same unmodified hardware.

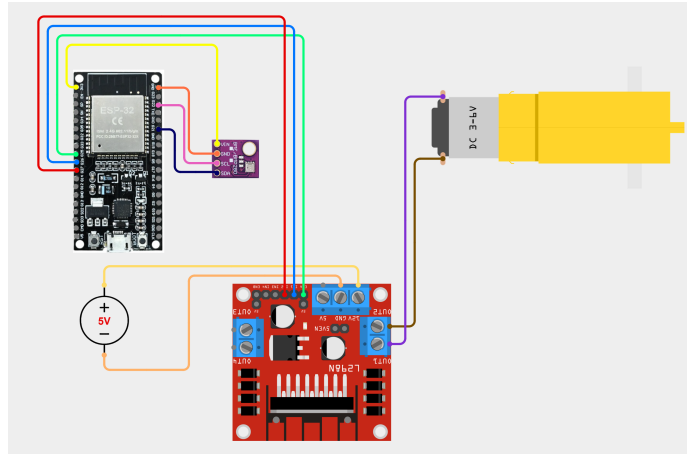


Figure 1: Wiring schematic (unchanged from Task 2.7/3.1).

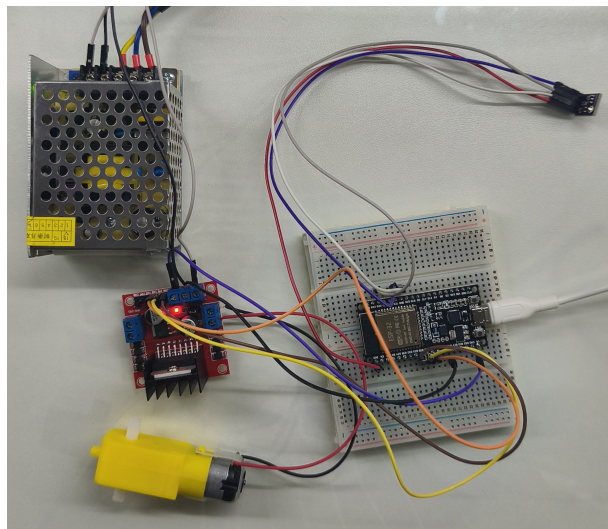


Figure 2: Assembled circuit (unchanged from Task 2.7/3.1).

4. Firebase Project Setup

- A new Firebase project (Dana-Task33) was created on the no-cost Spark plan.

- A Realtime Database instance was provisioned at : `https://dana-task33-default-rtdb.asia-southeast1.firebaseio.com/`, deliberately started in locked mode rather than test mode.
- A service-account private key was generated (Project settings → Service accounts → Generate new private key) and stored outside the git repository, alongside the existing Mosquitto password file, and excluded via `.gitignore`.
- The Node-RED palette `@gogovega/node-red-contrib-firebase-realtime-database` (v0.8.2, with its `@gogovega/firebase-config-node` v0.3.5 dependency) was installed, adding **Firebase in**, **Firebase get**, **Firebase out**, and **On Disconnect** nodes to the palette. Only **Firebase out** is used in this task.

Locked mode over test mode

Firebase's Realtime Database security rules govern *client-side* access – requests made with a public API key or no credentials at all. The Node-RED nodes in this task authenticate with the service-account private key (the Admin SDK), which bypasses security rules entirely regardless of what they say. Locked mode (`"read": false, "write": false`) was therefore chosen instead of the test-mode default: it gives Node-RED the exact same write access via the admin key, while closing off the database to any client-side request that isn't using that key – a strictly better security posture at no functional cost for this task.

Creating a firebase project:

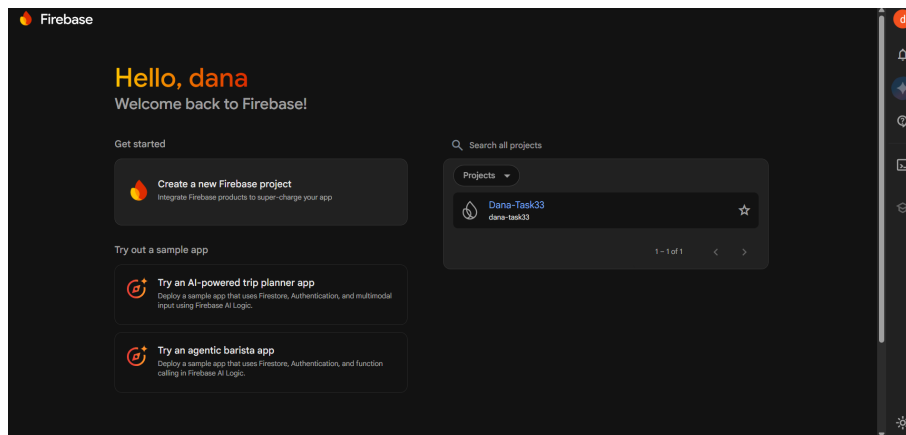


Figure 3: 1-create a new project

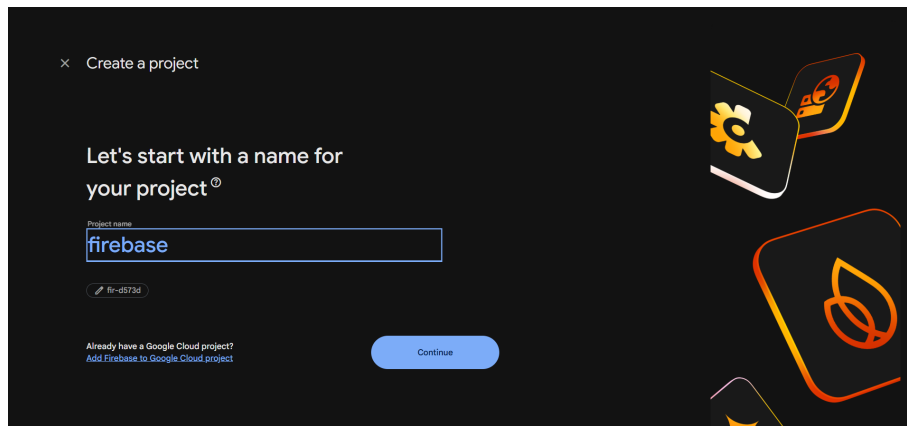


Figure 4: 2-name your project

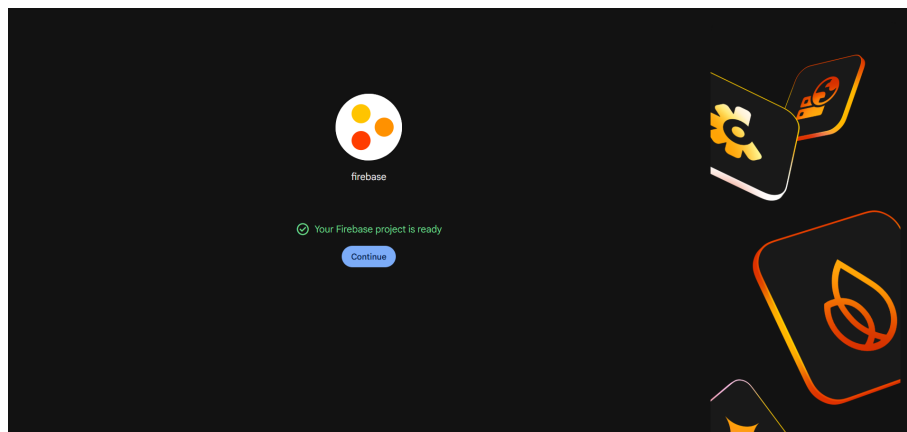


Figure 5: 3-wait for firebase to create the project

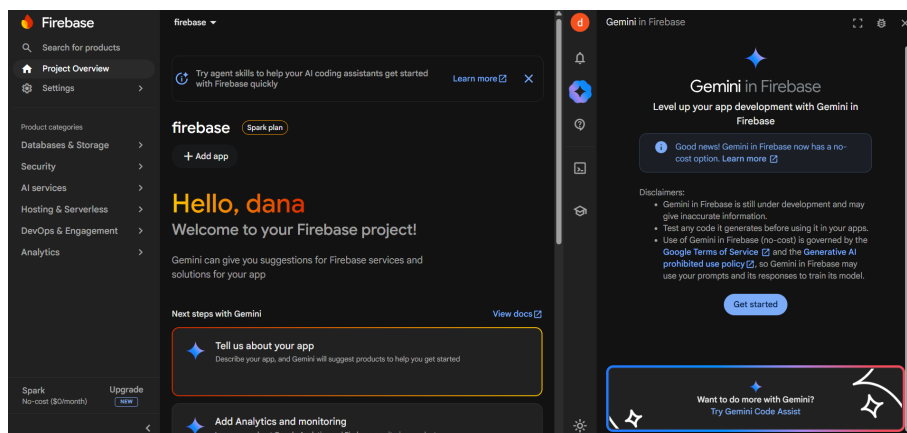


Figure 6: 4-project is ready

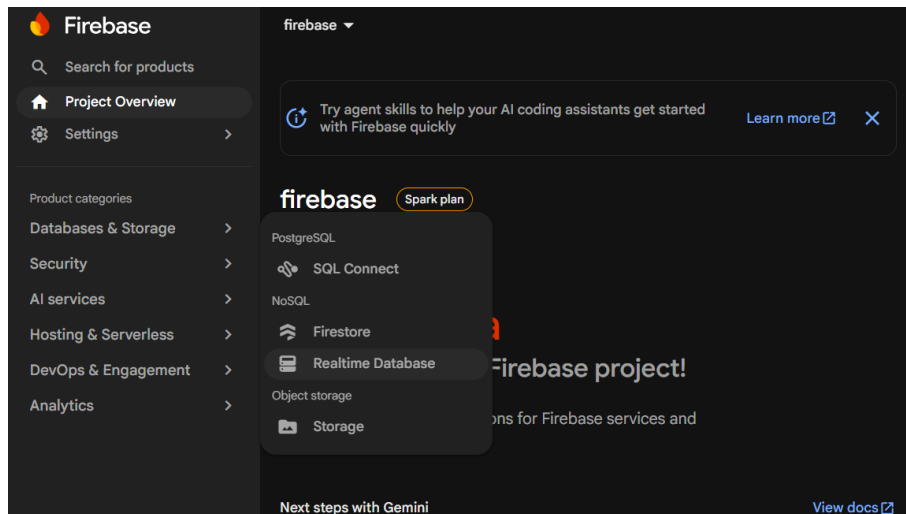


Figure 7: 5-create a realtime database inside your project

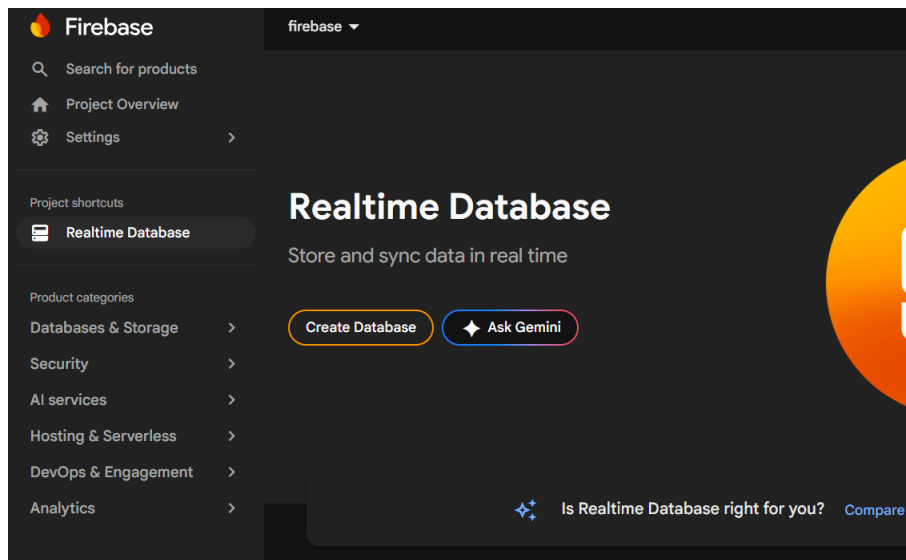


Figure 8: 6-click on the create database button

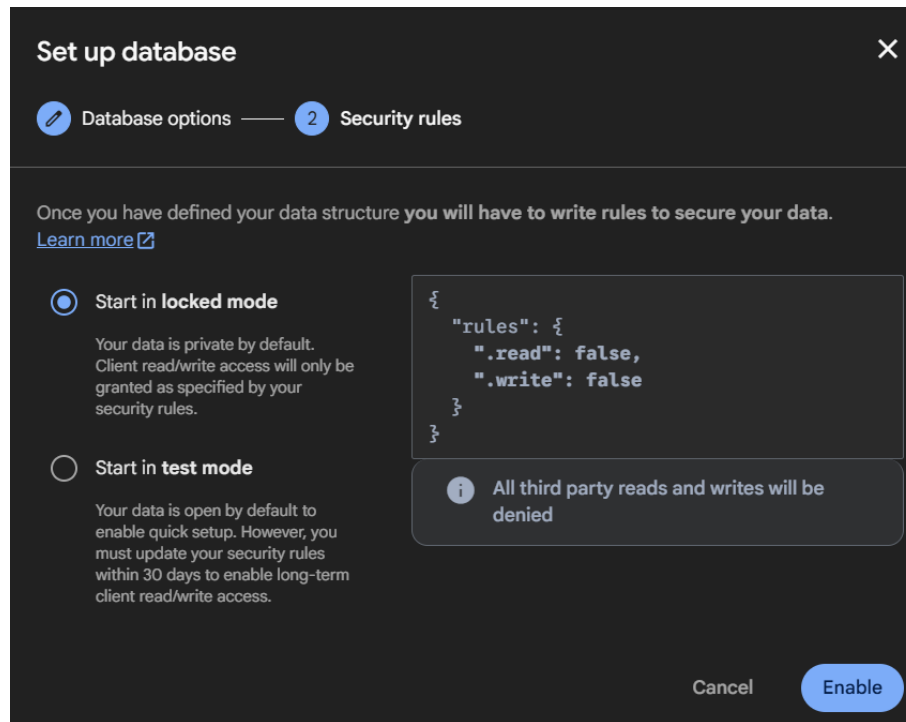


Figure 9: 7-set up database mode

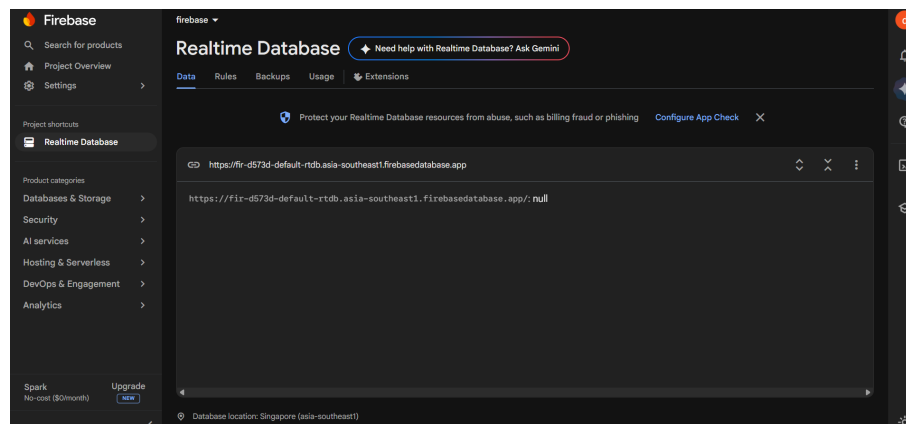


Figure 10: 8-Your database is ready

5. Node-RED Flow Additions

All new nodes live on the same tab (Task 3.1 - ESP32 Dashboard) as the original flow, so a single `flows.json` export contains both the Task 3.1 dashboard and the Task 3.3 Firebase branch together.

Tap points – no duplicated logic

Rather than re-subscribing to MQTT or re-implementing validation, each Firebase writer taps an existing node's *output*:

Firebase field	Tapped from (existing Task 3.1 node)
temperature, humidity, pressure	Output of the three <code>string to int change</code> nodes – the same validated, numeric values already driving the dashboard gauges and charts, <i>not</i> the raw MQTT string payloads
motor.state, motor.direction, motor.speed	Output of <code>motor state in</code> , <code>motor direction in</code> , <code>motor speed in mqtt in</code> nodes
status	Output of <code>Connection monitor</code> – the existing dual-signal (LWT + watchdog) function, <i>not</i> a second read of the raw <code>device/esp32/status</code> LWT topic

Tapping the *converted* sensor values rather than the raw MQTT payloads directly avoids repeating the Task 3.1 payload-type bug, where an un-converted numeric string silently broke `ui_chart`. Tapping `Connection monitor`'s output rather than the raw LWT topic avoids two independent writers racing to set the same status field – if the raw LWT topic were wired in separately, a stale “online” retained message and a fresh “stale” watchdog verdict could arrive in either order, non-deterministically reintroducing the exact lagging-state problem the watchdog exists to prevent.

Current-state writer

A single function node, `Merge to Firebase current`, receives all six tap points above. It keeps a running object in flow context keyed by the incoming message's MQTT topic (or, for the connection-status message, by its `payload.status` field), stamps it with `lastUpdated`, and forwards the complete object downstream on every incoming message.

```

1 let state = flow.get('firebaseCurrent') || {};
2
3 switch (msg.topic) {
4   case 'sensor/bme280/temperature':
5     state.temperature = msg.payload;
6     break;
7   case 'sensor/bme280/humidity':
8     state.humidity = msg.payload;
9     break;
10  case 'sensor/bme280/pressure':
11    state.pressure = msg.payload;
12    break;
13  case 'motor/status/state':
14    state.motor = state.motor || {};
15    state.motor.state = msg.payload;
16    break;
17  case 'motor/status/direction':
18    state.motor = state.motor || {};
19    state.motor.direction = msg.payload;
20    break;
21  case 'motor/status/speed':
22    state.motor = state.motor || {};
23    state.motor.speed = Number(msg.payload);

```

```

24     break;
25     default:
26         if (msg.payload && typeof msg.payload === 'object' && msg.payload.status) {
27             state.status = msg.payload.status.toLowerCase();
28         } else {
29             return null;
30         }
31     }
32
33     state.lastUpdated = Date.now();
34     flow.set('firebaseCurrent', state);
35
36     msg.payload = state;
37     return msg;

```

Listing 1: Merge to Firebase current – function node body

The output feeds a **Firestore out node (Database)** configured with **Method: SET** and a static **Path: current** – a full overwrite of `/current` on every call, never a partial write to a sub-field. This matters: a **Set** only replaces the exact path it targets, so writing the *entire* merged object to `/current` each time is what keeps every field present; writing only a changed sub-field (e.g. setting `/current` directly with just `{temperature: ...}`) would silently wipe out every other field at that level.

History writer

A repeating **inject node (History interval (15s))** fires every 15 seconds – the interval chosen from the task’s allowed 10–30s range – into a second **function node, Snapshot for history**, which reads the *same* shared state rather than re-deriving it from MQTT a second time:

```

1  const state = flow.get('firebaseCurrent');
2  if (!state) {
3      return null;
4  }
5  msg.payload = Object.assign({}, state, {
6      timestamp: Date.now()
7  });
8  return msg;

```

Listing 2: Snapshot for history – function node body

Its output feeds a second **Firestore out node (Database History)**, sharing the same **Firestore config** node as the current-state writer, configured with **Method: PUSH** and a static **Path: history**. **Push** (rather than **Set**) is what makes each 15-second tick land as its own new, chronologically-sortable key under `/history` instead of overwriting the previous entry.

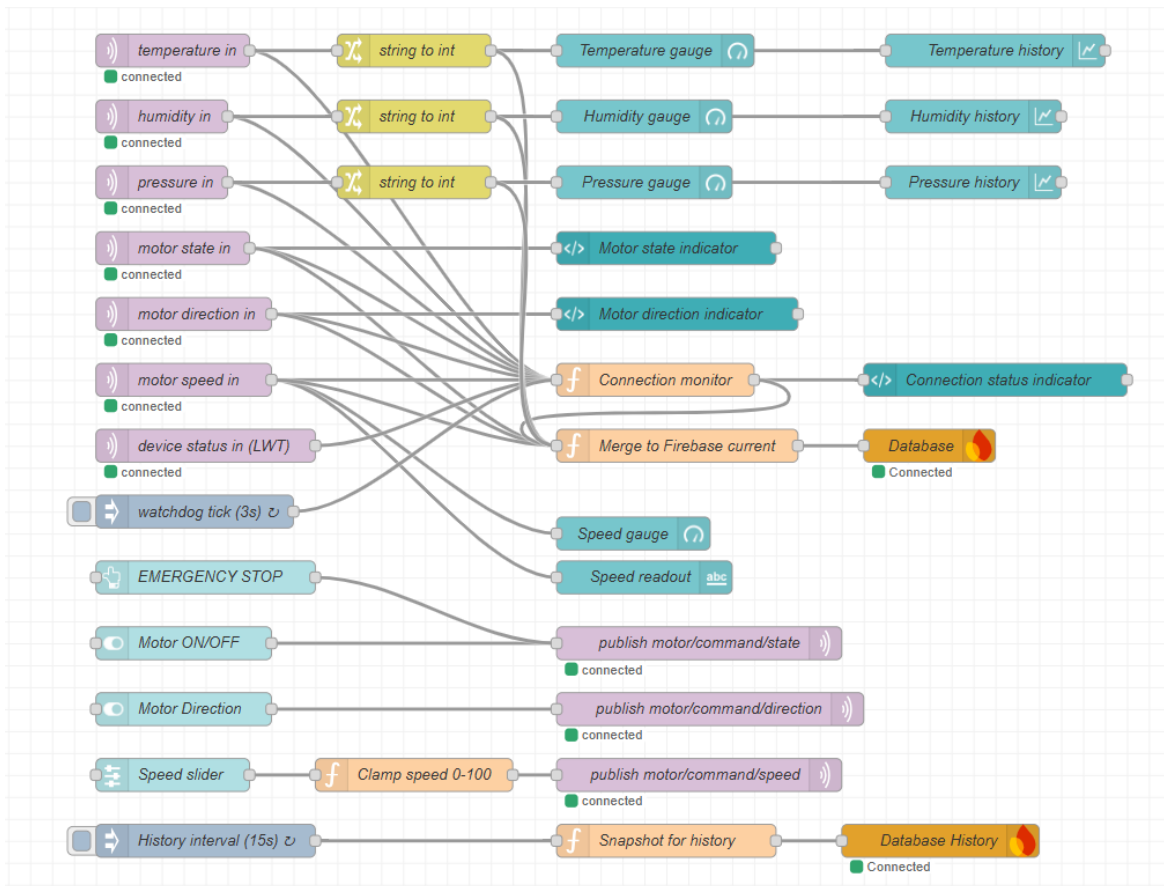


Figure 11: Node-RED canvas.

6. Data Structure

```

1 /current
2   temperature: 23.13
3   humidity: 39.38
4   pressure: 895.72
5   motor:
6     state: "0"
7     direction: "forward"
8     speed: 0
9   status: "online"
10  lastUpdated: 1786526842309
11
12 /history
13 -0zpI0c5j37hCJZvMwC5:
14   temperature, humidity, pressure, motor, status, lastUpdated,
15   timestamp: 1786526842309
16 -0zpI0d...: { ...next snapshot, 15s later... }

```

Listing 3: Firebase Realtime Database tree

Node	Write method	Behavior
<code>/current</code>	Set (full overwrite)	Always reflects “right now.” One object, replaced in full on every tracked change; nothing to compute when reading it.
<code>/history</code>	Push (auto-keyed append)	Append-only log. A new, uniquely-keyed snapshot is added every 15 seconds; existing entries are never touched.

7. Non-Blocking Behavior

The Firebase Realtime Database client used by `@gogovega/node-red-contrib-firebase-realtime-database` issues writes asynchronously over its own persistent connection, independent of Node-RED’s main flow-execution path. Because every Firebase node in this task is wired as a *branch* off an existing tap point – never inserted in series between an incoming MQTT message and the dashboard widgets that already consume it – the dashboard’s gauges, charts, and indicators receive their messages exactly as they did in Task 3.1, unaffected by whether or how quickly the Firebase write completes. This was confirmed by observing continued live gauge and chart updates on the dashboard throughout testing, including while intentionally disconnecting the ESP32 to trigger the offline test in Section .

8. Debugging Journal: The Silent History Branch

Symptom

After the current-state writer was confirmed working – `/current` populated correctly in the Firebase console, matching the dashboard – the history branch was deployed and left running for several minutes. `/current` kept updating live. `/history` never appeared in the console tree at all: not empty, not partially populated, simply absent.

Hypothesis 1 (ruled out): inject node not repeating

The `History interval (15s)` inject node’s **Repeat** field was checked first, since an inject node left on the default `none` fires exactly once and never again. It was confirmed set to `interval / 15 seconds` – ruled out.

Hypothesis 2 (ruled out): Firebase out node misconfigured

The `Database History` node’s `Path` field showed a red validation outline in the editor at one point, which looked like a strong candidate for the actual fault. Retyping the path and clicking away cleared the outline with no change in behavior – a stale UI validation state, not a real configuration error. This was ruled out by the absence of any error indicator on the deployed node itself.

Hypothesis 3 (the real root cause): node-scoped context

The two function nodes – `Merge to Firebase current` and `Snapshot for history` – were written under the assumption that `context.get('firebaseCurrent')` would return whatever the other node had most recently `set`. In Node-RED, however, the plain `context` object is scoped *per node* by default: each function node has its own private context store, invisible to every other node, including other function nodes on the same tab. `Snapshot for history` was therefore calling `context.get('firebaseCurrent')` on its own, permanently-empty store, hitting its own `if (!state) return null;` guard on every single tick – silently discarding the message before it ever reached the `Firebase out` node.

```

1 // Merge to Firebase current
2 context.set('firebaseCurrent', state); // writes to THIS node's private store
3
4 // Snapshot for history (a different node)
5 const state = context.get('firebaseCurrent'); // reads from ITS OWN, empty store
6 if (!state) { return null; } // ...and always hits this

```

Listing 4: The bug – each function node has its own private context

Fix

Both functions were switched from node-scoped `context` to flow-scoped `flow`, which is shared by every node on the same tab:

```

1 // Merge to Firebase current
2 flow.set('firebaseCurrent', state);
3
4 // Snapshot for history
5 const state = flow.get('firebaseCurrent');

```

Listing 5: The fix

Confirmation

After redeploying and waiting one 15-second interval, a hard refresh of the Firebase console showed a new `history` node at the database root, containing a uniquely-keyed entry with the full merged state and a `timestamp` field.

Takeaway: this bug produced no error anywhere – no red triangle on either function node, no failed-deploy warning, no exception in the debug sidebar. `return null;` is a deliberate, silent no-op in Node-RED, which is exactly the right behavior for an intentional guard clause and exactly the wrong behavior to debug by watching for errors. The fault was only found by treating the guard clause itself as a suspect and checking *which* context each node was actually reading from, rather than assuming two nodes named similarly (`context.get/ context.set`) were talking to the same store – the same discipline of verifying data is actually arriving, rather than trusting that a wire existing means data is flowing, that resolved the blank-chart bug in Task 3.1.

9. Results & Verification

Live current state matches the dashboard

With the ESP32 connected and reporting, `/current` in the Firebase console reflected the same values shown on the Node-RED dashboard gauges and indicators, updating on the same cadence as the underlying MQTT publishes.

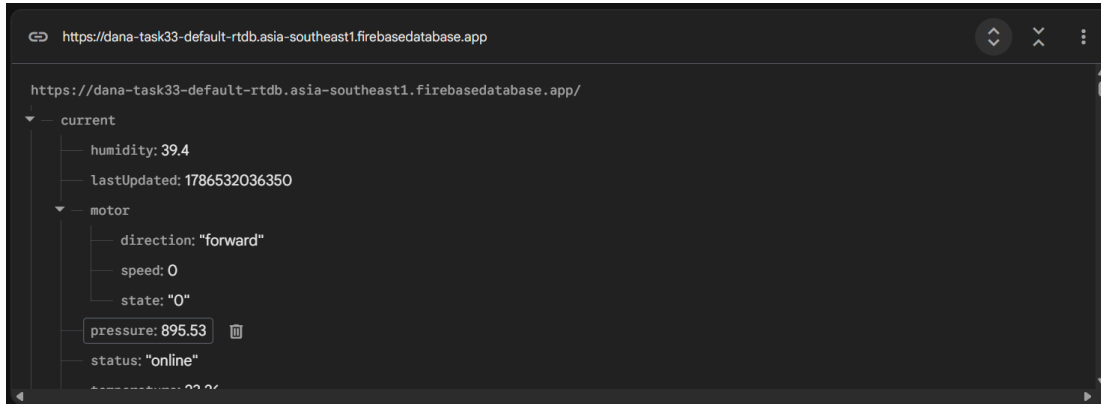


Figure 12: `/current` in the Firebase console: validated numeric sensor values, nested motor state, and live connection status.

History accumulating over time

After the flow-context fix, `/history` accumulated one new, uniquely-keyed entry every 15 seconds, each containing a full state snapshot plus its own `timestamp`.

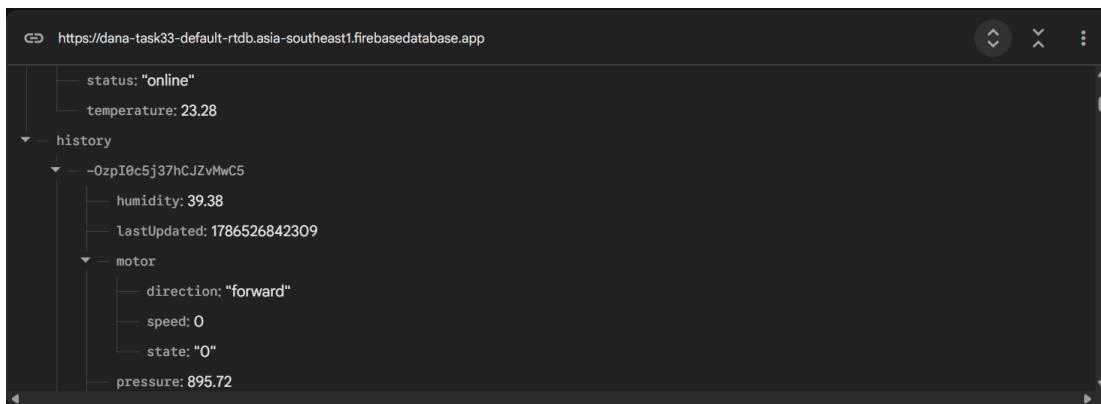


Figure 13: `/history` in the Firebase console: auto-generated push keys, each holding a complete timestamped snapshot.

Offline detection reaching Firebase

With the ESP32 running normally, `/current/status` read `"online"`. The ESP32 was then disconnected (power/Wi-Fi), and within a few seconds – driven by the same retained LWT and `Connection monitor` watchdog logic built in Task 3.1 – the value was confirmed to update to `"offline"` in the Firebase console, without needing a manual refresh of the writer or any change to the detection logic itself.

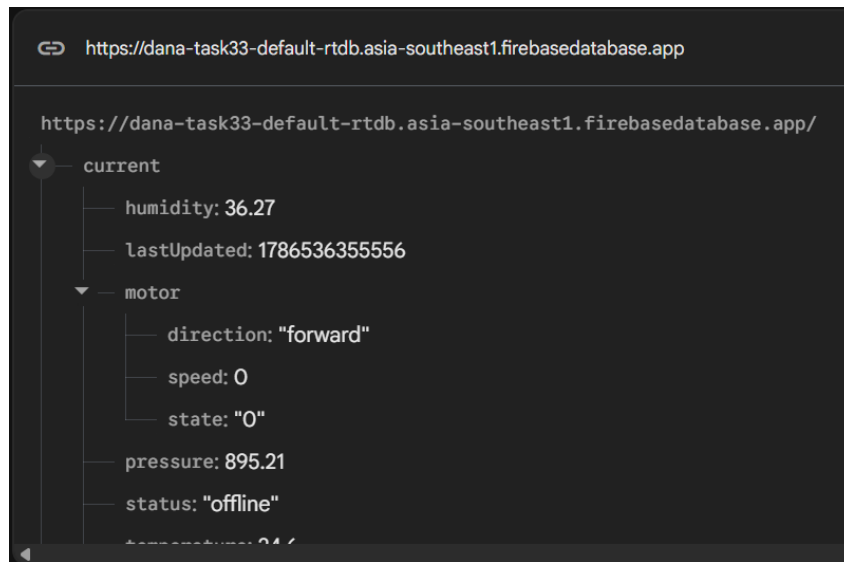


Figure 14: /current/status showing "offline" in the Firebase console shortly after disconnecting the ESP32.

Dashboard unaffected

Throughout all of the above, the Task 3.1 Node-RED dashboard – gauges, charts, motor indicators, and controls – continued to update and respond exactly as it did before the Firebase branch existed.

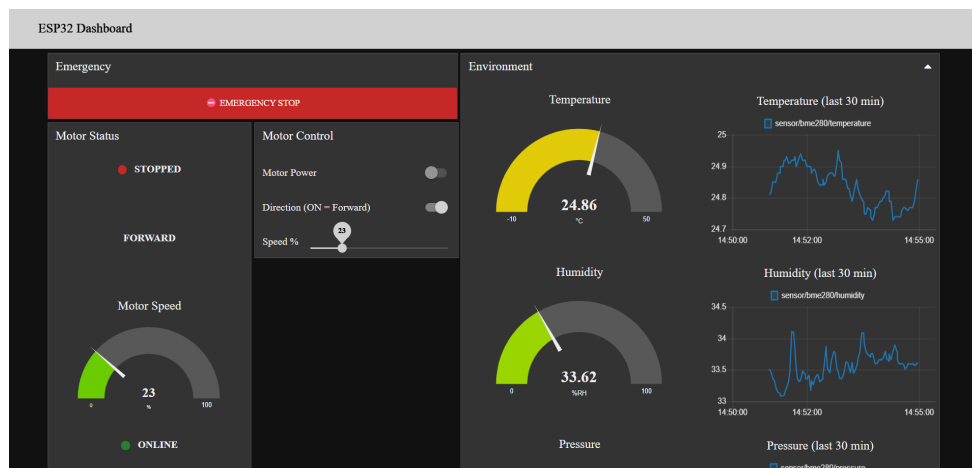


Figure 15: Node-RED dashboard operating normally alongside the Firebase writes, confirming the additive branch does not block or slow the existing UI.

10. Design Notes – Efficient Choices

1. **One shared snapshot, two write patterns.** Rather than wiring six separate Firebase nodes (one per value) or re-deriving history from raw MQTT messages, a single cached object in flow

context serves both writers – **Set** for “now” and **Push** for “over time” – eliminating any possibility of the two views drifting out of sync with each other.

2. **Tap validated data, not raw payloads.** Sourcing sensor values from the *output* of the existing **string to int** nodes, rather than the raw **mqtt in** payloads, means Firebase can never show a value the dashboard itself would have silently rejected – directly avoiding a repeat of Task 3.1’s payload-type bug in a new location.
3. **Reused, not re-derived, availability logic.** Tapping **Connection monitor**’s already-computed **ONLINE/STALE/OFFLINE** verdict, instead of reading the raw **LWT** topic a second time, guarantees Firebase and the dashboard can never disagree about connection state – there is only ever one place that decision is made.
4. **Locked database rules over test mode.** Since the Admin SDK bypasses Realtime Database security rules entirely, locking the database down costs nothing functionally while closing off any client-side access that isn’t using the service-account key.
5. **Firebase as a pure branch, never in series.** Every Firebase node consumes a message that some other node has already delivered to the dashboard; none of them sit between an MQTT input and a dashboard widget. This is what keeps the Firebase writes’ latency, and any transient connectivity issue on the Firebase side, from ever being able to affect the local dashboard’s responsiveness.

11. Known Limitations & Considerations

- **Firebase credentials are machine-local.** The service-account private key lives outside the repository on the machine running Node-RED, the same way the Mosquitto password file is already handled; it is not part of the exported **flows.json** and must be re-supplied if the flow is imported elsewhere.
- **History interval is fixed, not adaptive.** The 15-second snapshot cadence is a flat timer, independent of whether any underlying value actually changed in that window – an unchanged reading is logged just as a changed one is.
- **No retry/backoff on Firebase write failure.** If the Firebase connection drops momentarily, an in-flight **Set** or **Push** is not explicitly retried by this flow; the next scheduled write (next MQTT message, or next 15-second tick) simply supersedes it.
- **Locked-mode rules assume the Admin SDK is the only writer.** Because rules are bypassed entirely by the service account, this setup does not by itself protect against a second, misconfigured Node-RED instance sharing the same key.
- **flow context is in-memory.** Like **ui_chart**’s in-memory point buffer noted in Task 3.1, the cached **firebaseCurrent** object in flow context does not survive a Node-RED restart – **/current** simply repopulates from the next incoming MQTT messages, with no data loss to the database itself.

12. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Unchanged Task 2.7/3.1 firmware
BME280 environmental sensor module	1	I2C, address 0x76
L298N (or equivalent) H-bridge motor driver	1	Driven via IN1/IN2/ENA
DC motor	1	Bidirectional, speed-controlled via PWM
PC running Mosquitto 2.1.2	1	Same broker host as Task 2.7/3.1
PC running Node.js + Node-RED + node-red-dashboard	1	Same machine as the broker; also hosts the new Firebase nodes
@gogovega/node-red-contrib-firebase-realtime-database	v0.8.2	Provides Firebase out used in this task
@gogovega/firebase-config-node	v0.3.5	Shared config-node dependency
Firebase project (Spark / no-cost plan)	1	Realtime Database, locked-mode rules
Firebase service-account key	1	Stored outside the repo; Admin SDK authentication
Web browser	2	Node-RED dashboard client (:1880/ui) and Firebase console

13. Future Improvements

- **Change-triggered history entries** in addition to the fixed 15-second timer, so a value that actually moved is logged immediately rather than waiting for the next tick.
- **Firebase Security Rules scoped to specific service accounts or authenticated users**, for a deployment where more than one writer might eventually share the database.
- **Automatic pruning or export of /history** to a colder store once it grows large, so the Realtime Database's working set stays small and fast to sync.
- **Retry/backoff wrapper around Firebase writes** so a transient network drop doesn't silently skip a scheduled **Set** or **Push**.
- **Firebase Cloud Messaging or a status webhook** triggered directly off the **Connection** monitor transition to **OFFLINE**, for push notification of a real disconnect rather than requiring someone to be watching the console.